

Trabajo Práctico Especial: Servidor Proxy SOCKS v5

Integrantes

- Manuel Suñol - 64138
- Augusto Felipe Ospal Madeo - 63669
- Valentín Ontivero - 60034
- Mateo Stella - 65780

1. Índice

1. Índice
2. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas
3. Problemas encontrados durante el diseño y la implementación
4. Limitaciones de la aplicación
5. Posibles extensiones
6. Conclusiones
7. Ejemplos de prueba
8. Guía de instalación detallada y precisa
9. Instrucciones para la configuración
10. Ejemplos de configuración y monitoreo
11. Documento de diseño del proyecto

2. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas

Aplicaciones Desarrolladas

Se desarrollaron dos aplicaciones principales:

1. **Servidor Proxy SOCKS v5** ( `server`): Un servidor concurrente basado en sockets no bloqueantes multiplexados mediante `pselect()`. Implementa la máquina de estados necesaria para parsear los requests definidos en el [RFC 1928](#) y el [RFC 1929](#) (autenticación usuario/contraseña). Soporta IPv4, IPv6 y resolución asincrónica de FQDNs.
2. **Cliente de Monitoreo** ( `client`): Una interfaz de línea de comandos interactiva que permite a los administradores conectarse al puerto de *management* del servidor para visualizar métricas, consultar el registro de accesos (access log) y administrar usuarios en tiempo de ejecución.

Diseño del Protocolo de Monitoreo (Estilo RFC)

Para satisfacer el requerimiento de gestión en tiempo de ejecución, se diseñó el protocolo **ProtoSocks Management Protocol (PSMP)**.

Justificación de Diseño: Se optó por un protocolo **basado en texto** sobre **TCP** (típicamente en el puerto 8080) debido a su simplicidad, facilidad para depurar (es compatible con netcat o telnet) y la falta de requerimientos estrictos de ancho de banda para las tareas de administración. La comunicación es estrictamente del tipo Solicitud-Respuesta (Request-Reply), donde el cliente toma la iniciativa.

Transporte y Codificación:

- **Transporte:** TCP.
- **Codificación:** ASCII / UTF-8.
- **Delimitadores:** Cada mensaje de solicitud y de respuesta debe terminar con un salto de línea (LF, `\n`, ASCII 10). Se toleran y descartan los retornos de carro (CR, `\r`, ASCII 13) previos al LF.

Autenticación: Al establecerse la conexión TCP, el servidor asume un estado NO AUTENTICADO. Cualquier comando (excepto `AUTH`) enviado en este estado responderá con un error. El cliente debe enviar el token de administración provisto al iniciar el servidor.

Formato de Respuesta General: Las respuestas del servidor comienzan con un indicador de éxito o error:

- Éxito simple: `+OK\n`
- Error: `-ERR <mensaje descriptivo>\n`

Para los comandos que retornan múltiples datos (listas o métricas), el servidor primero envía líneas con la información solicitada y siempre finaliza la transmisión exitosa con un `+OK\n`.

Comandos Soportados:

1. Autenticación (AUTH)

- **Solicitud:** `AUTH <token>\n`
- **Respuesta Exitosa:** `+OK\n`
- **Respuesta Fallida:** `-ERR Invalid token\n` (cierra la conexión).

2. Métricas (METRICS)

- **Solicitud:** `METRICS\n`
- **Respuesta:**

```
historic_connections:<entero>\nconcurrent_connections:<entero>\nbytes_transferred:<entero>\n+OK\n
```

3. Agregar Usuario (ADD-USER)

- **Solicitud:** `ADD-USER <username> <password>\n`
- **Respuesta Exitosa:** `+OK\n`
- **Respuestas Fallidas:** `-ERR Invalid arguments\n`, `-ERR Could not add user\n` (ej. límite alcanzado).

4. Eliminar Usuario (DEL-USER)

- **Solicitud:** DEL-USER <username>\n
- **Respuesta Exitosa:** +OK\n
- **Respuestas Fallidas:** -ERR Invalid arguments\n , -ERR Could not remove user\n (ej. usuario no existe).

5. Listar Usuarios (LIST-USERS)

- **Solicitud:** LIST-USERS\n
- **Respuesta:**

```
users:<cantidad>\n
<username_1>\n
<username_2>\n
+OK\n
```

6. Registro de Accesos (ACCESS-LOG)

- **Solicitud:** ACCESS-LOG\n
- **Respuesta:**

```
access_log:<cantidad>\n
<timestamp> user=<user> dst=<destino> port=<puerto> status=<estado>\n
+OK\n
```

3. Problemas encontrados durante el diseño y la implementación

- **Gestión de memoria de buffers:** Definir un modelo de *zero-copy* o minimizar copias de buffers entre sockets no bloqueantes fue un desafío. Se adoptó un diseño de buffer circular por cliente y estado (estructuras `raw_buff_a` y `raw_buff_b` asignadas dinámicamente o por un pool de objetos).
- **Resolución de Nombres No Bloqueante (DNS):** La función POSIX `getaddrinfo()` es intrínsecamente bloqueante. Para no violar el principio de un solo hilo I/O del servidor, la resolución se delegó a *worker threads* con `pthread_create` . Esto introdujo problemas de sincronización, resueltos utilizando `pthread_kill` o señales y notificaciones del selector cuando el hilo termina de resolver.
- **Techo de `FD_SETSIZE` :** Durante las pruebas iniciales, el uso de la API clásica `pselect(2)` limitaba severamente el servidor a ~500 conexiones (dado que `FD_SETSIZE` por default es 1024, y cada conexión requiere dos FDs).
- **Problema de acoplamiento de responsabilidades en la recolección de métricas:** Inicialmente, planteamos almacenar las variables globales de contabilidad de estadísticas (como las conexiones históricas o el conteo de bytes leídos) directamente en la máquina de estados del proxy (`socks5nio.c`). Sin embargo, al momento de querer implementar el comando METRICS en el servidor de monitoreo (`monitor.c`), nos topamos con un fuerte acoplamiento: el monitor debía conocer la estructura interna del proxy o acceder a variables globales del archivo de SOCKS5, violando la separación de capas y dificultando la mantenibilidad futura si decidíamos modificar el protocolo de control. Decidimos crear un módulo intermedio dedicado únicamente a las estadísticas (`metrics.h` y `metrics.c`). En lugar de que `socks5nio.c` exponga sus variables internas. El proxy simplemente "notifica" eventos

mediante una API clara (como `metrics_register_new_connection()` o `metrics_register_bytes_transferred(n)`), y el monitor consulta la información mediante getters puros.

4. Limitaciones de la aplicación

1. **Límite Teórico de Concurrency:** Debido al uso de macros estándar `fd_set` y la función `pselect(2)`, el servidor soporta como máximo ~510 conexiones concurrentes y simultáneas, estando a merced de `FD_SETSIZE = 1024`.
2. **Escalabilidad del Access Log:** El historial de accesos está limitado a un tamaño estático (`ACCESS_LOG_MAX_ENTRIES`) para evitar un consumo desmedido e impredecible de memoria, comportándose como un buffer circular (se sobrescriben los más antiguos).
3. **Sockets Pasivos solo IPv4:** Si bien el proxy puede hacer conexiones salientes (hacia el *origin server*) tanto en IPv4 como en IPv6, los puertos pasivos de escucha locales del servidor y de administración están fijos en la familia `AF_INET` (IPv4).

5. Posibles extensiones

1. **Migración a `epoll(2)`:** Para derribar la limitante artificial de ~510 conexiones impuesta por `pselect(2)` y `FD_SETSIZE`, la arquitectura podría evolucionar a utilizar Linux `epoll`. Esto permitiría escalar a miles de conexiones concurrentes sin pérdida notable de performance.
2. **Persistencia de Configuración y Métricas:** Dotar al servidor de la capacidad de guardar/restaurar sus métricas y la tabla de usuarios autorizados en un archivo de configuración `.conf` local, en lugar de depender únicamente de los flags en tiempo de arranque.
3. **Soporte IPv6 pasivo:** Permitir escuchar y recibir conexiones entrantes (clientes SOCKS) desde direcciones IPv6 puras (`AF_INET6`).

6. Conclusiones

El desarrollo de *ProtoSocks* fue exitoso en demostrar el uso fundamental del multiplexado I/O de bajo nivel (`pselect`), probando que es factible atender cientos de conexiones concurrentes en C manejando adecuadamente los recursos desde un hilo único principal.

El modelo de **Máquina de Estados Finita** (FSM) resultó crucial; aislar la complejidad de SOCKS v5 (HELLO, AUTH, REQUEST) en pequeños fragmentos de código desacoplados permitió depurar fallas y manejar fácilmente lecturas y escrituras parciales, garantizando que el servidor nunca se bloquee frente a un cliente lento (*slowloris*).

Respecto al estrés, se concluye empíricamente que la delegación de `getaddrinfo` a hilos temporales es el cuello de botella más sensible de esta arquitectura cuando ocurren cientos de *requests* de DNS de forma simultánea.

7. Ejemplos de prueba

Para comprobar que el proxy funciona y cursa tráfico hacia Internet:

1. Navegación HTTP a Google (vía cURL):

```
► curl -v -x socks5://juan:password123@127.0.0.1:1080 http://www.google.com
```

2. Descarga directa de archivos pesados:

```
► wget -e use_proxy=yes -e SOCKS_PROXY=127.0.0.1:1080 http://speedtest.tele2.net/10MB.zip
```

(Luego, se puede revisar mediante el cliente de monitoreo el contador de "bytes_transferred" y constatar el aumento).

3. Prueba de error DNS (Servidor Inexistente):

```
► curl -v -x socks5://juan:password123@127.0.0.1:1080 http://este-dominio-no-existe-123.com
```

(Deberá devolver un error mapeado `SOCKS5 Host Unreachable`).

8. Guía de instalación detallada y precisa

El código fuente incluye todo lo necesario y no utiliza dependencias extrañas ajenas al ecosistema estándar POSIX/Linux.

1. Extraiga el código fuente.
2. Abra un emulador de terminal en el directorio raíz del proyecto (donde está el archivo ► `Makefile`).
3. Ejecute el comando:

```
► make
```

4. El sistema compilará todos los objetos y vinculará dos archivos binarios resultantes en el subdirectorio `bin/` :
 - `bin/server`
 - `bin/client`
5. (Opcional) Para limpiar el entorno de compilación en cualquier momento:

```
► make clean
```

6. (Opcional) Si se quiere modificar el proyecto también se puede ejecutar ► `format` con make para conservar el formato que se usó en el resto del proyecto. También se pueden usar las reglas ► `tidy` con make analizar el código c, y también ► `tidy-fix` para corregir algunas cosas automáticamente.

9. Instrucciones para la configuración

El servidor no requiere de archivos `.conf` . Toda su configuración se determina pasándole argumentos al invocar su binario. Si se lo ejecuta sin parámetros, el servidor toma las siguientes decisiones por defecto:

- **Proxy SOCKS5:** Puerto 1080, disponible en cualquier interfaz (`0.0.0.0`).
- **Management API:** Puerto 8080, restringido a la red local (`127.0.0.1`), token de autenticación: `admin123` .

Para modificar estos valores, se pueden emplear combinaciones de opciones. Las opciones se describen ejecutando `./bin/server -h` :

```
-h                Imprime la ayuda y termina.
-l <SOCKS addr>  Dirección donde servirá el proxy SOCKS.
-L <conf addr>   Dirección donde servirá el servicio de management.
-p <SOCKS port>  Puerto entrante conexiones SOCKS.
-P <conf port>   Puerto entrante conexiones configuracion.
-t <token>       Token de autenticación del servicio de management.
-u <name>:<pass>  Usuario y contraseña que puede usar el proxy (repetible).
-N              Deshabilita disectores de protocolo.
-v              Imprime información sobre la versión y termina.
```

10. Ejemplos de configuración y monitoreo

Levantar el servidor

Ejemplo: Levantar el servidor SOCKS en el puerto alternativo `9090` , el puerto de monitoreo en él `9091` con token seguro, y pre-cargar dos usuarios de entrada:

```
▶ ./bin/server -p 9090 -P 9091 -t "MiTokenSuperSecreto" -u "ricardo:clavetest" -u "profesor:12345"
```

Interacción y monitoreo con el CLI de cliente

En otra terminal, lanzamos el cliente administrador hacia los parámetros definidos:

```
▶ ./bin/client -P 9091 -t MiTokenSuperSecreto
```

Al abrirse la consola interactiva, el prompt será `admin>` . A continuación, una sesión de ejemplo:

Revisando usuarios activos:

```
admin> list-users
users:2
```

```
ricardo
profesor
+OK
```

Agregando un nuevo usuario:

```
admin> add-user pablito pass1234
+OK
```

Revisando las métricas en caliente:

```
admin> metrics
historic_connections:5
concurrent_connections:1
bytes_transferred:40960
+OK
```

Visualizando quién está navegando (Access Log):

```
admin> access-log
access_log:2
2026-07-12 18:45:01 user=ricardo dst=142.250.190.4 port=443 status=OK
2026-07-12 18:45:10 user=profesor dst=93.184.216.34 port=80 status=NETWORK_UNREACHABLE
+OK
```

Saliendo:

```
admin> exit
Disconnecting...
```

11. Documento de diseño del proyecto

Arquitectura General

El servidor adopta un patrón arquitectónico *Single-threaded Non-blocking I/O Event Loop*. En lugar de crear un hilo (thread) o proceso por cada cliente que se conecta (modelo One-Thread-Per-Client), toda la operatoria se engloba en el hilo principal de ejecución.

Componentes Clave

1. **El `fd_selector` (Multiplexor):** Es un wrapper escrito sobre `pselect()`. Oculta los detalles sucios del manejo de conjuntos de bits de FD (`fd_set`). Se le registran descriptores de archivo de interés y devuelve eventos (Read Ready / Write Ready).

2. **El Ciclo de Eventos:** Itera infinitamente sobre el selector. Cuando el selector advierte de un cambio en el socket pasivo principal, se invoca `accept()`. Al obtener el descriptor del cliente, este se registra en el selector asociándole un "Handler de Eventos" (`socks5_handler`).
3. **Máquina de Estados (State Machine FSM):** Cada conexión cliente contiene el struct `socks5` que aloja una `stm`. El hilo principal ejecuta devoluciones de llamada (*callbacks*) en función del estado actual. Un cliente transita `HELLO_READ -> HELLO_WRITE -> AUTH_READ -> AUTH_WRITE -> REQUEST_READ -> RESOLVING -> CONNECTING -> REQUEST_WRITE -> COPY`. Si un estado solo lee partes del mensaje (debido a buffers vacíos/lLENOS del kernel), guarda su progreso en el *parser* interno y retorna, entregando de vuelta el control al Event Loop para atender al próximo cliente.
4. **Resolución DNS y Worker Threads:** Dado que `getaddrinfo` detendría todo el Event Loop bloqueando a todos los demás clientes, `socks5nio.c` delega esta única llamada a un *worker thread* en el estado `RESOLVING`. Cuando el thread resuelve el host, interactúa con el selector para emitir un evento especial "Block Ready", que retoma la máquina de estado del cliente y prosigue a conectarse (`CONNECTING`) a la IP remota.
5. **Subsistema de Management:** Un handler separado, pero inserto en el mismo Event Loop general (`monitor_handler`). Comparte el espacio de memoria del servidor, lo que le permite alterar arreglos y leer contadores compartidos (las métricas o `user_store`) con máxima eficiencia sin requerir semáforos ni Mutexes (puesto que todas las escrituras y lecturas las hace iterativamente el mismo y único hilo principal, garantizando aislamiento secuencial).

12. Bibliografía

Usamos principalmente el libro [Kerrisk, M. \(2010\). The Linux programming interface: A Linux and UNIX system programming handbook. No Starch Press](#). Muchas veces citamos directamente qué página usamos para alguna función particular. Al ser el unico libro que usamos, si en algún momento decimos que nos basamos del libro nos estamos refiriendo a este.